

# Chapter 56. MLOps: Making Models Survive Production

Versioning, tracking, packaging, serving, deployment patterns,  
three-layer monitoring, measured drift, retraining, and incidents.

Draft chapter

# Contents

## Chapter 56. MLOps: Making Models Survive Production

Why this matters

In plain English

56.1 What breaks after launch

56.2 Versioning: "the model" is five things

56.3 Experiment tracking

56.4 Packaging: what travels with the model

56.5 Serving

56.6 Deployment patterns

56.7 Monitoring, in three layers

56.8 Drift, measured

56.9 Retraining

56.10 When it goes wrong

56.11 How far up the ladder to climb

Common mistakes and how to spot them

In the real world: the fortnight the line got worse and nobody knew

Tools

The project: run a model for six months in an afternoon

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

# Chapter 56. MLOps: Making Models Survive Production

Part VI — Production ML, Generative AI & MLOps

## Chapter at a glance

**You will learn to:** name the four ways a working model stops working · version the five things that make up "the model" · track experiments so twelve runs can be compared instead of remembered · package a model with the metadata that has to travel with it · serve it behind a real HTTP endpoint with validation, versioning, and a log line monitoring can read · choose between batch, online, shadow, canary, and blue-green · monitor in three layers, and know which layer sees which failure · compute drift with PSI and the KS statistic, and see why drift alarms and model decay are not the same thing · decide when to retrain, and what to check before shipping the retrain · run an incident and write the post-mortem · and judge how far up the MLOps maturity ladder your company should actually climb.

**Before you start:** Chapter 53 (the defect model this chapter operates), Chapter 29 (tested, packaged Python), Chapter 34 (the command line, HTTP, exit codes), Chapter 32 (CI, and the habit of testing a pipeline), Chapter 38 (evaluation).

**Time needed:** 16–20 hours, spread over three weeks.

**Tools:** Python 3.12, scikit-learn, FastAPI and uvicorn, MLflow, joblib, SciPy. All installed locally; no cloud account.

**Practice data:** six months of Riverstone's moulding line, built by `simulate_production.py`: 12,000 parts over 24 weeks, with the lamps replaced in week 8 and a new mould producing an unlabeled defect from week 16.

---

## Why this matters

Chapter 53 ended with a defect model that catches 97.5% of defects at a cost the plant manager agreed to. That was the easy part. The hard part is the next six months, and it fails in four ways that have nothing to do with modeling:

- **The world moves.** New lamps, a new mould, a new supplier's resin. The model was trained on a world that no longer exists.
- **Nobody notices.** A model that has quietly got worse looks exactly like a model that is fine, unless someone is measuring.
- **Nobody can reproduce it.** Six months later, "which data was this trained on?" has no answer, so the model cannot be rebuilt, audited, or improved.
- **Nobody can roll it back.** A retrain ships, quality drops, and there is no previous version to return to.

This chapter is the discipline that fixes all four, and it is the difference between a demo and a system. It is also, in most companies, the work that goes to the person who can *both* train a model and run software, which is Chapter 8's analytics-engineer-to-ML-engineer path and a significant salary step.

---

## In plain English

---

**A model is a perishable good, and MLOps is the cold chain.**

A trained model is a set of numbers that encode how the world looked on the day the training data was collected. Deploy it and it starts aging immediately, because the world doesn't stand still: lighting changes, products change, customers change, and the model doesn't.

The cold chain has four parts, and you need all four:

- **Labels on the box.** Which data, which code, which parameters, which day. Without them you cannot reproduce, compare, or roll back.
- **A thermometer.** Monitoring, in layers: is the service up, does the incoming data look like what we trained on, and is the model still right? Each layer catches a different failure, and the last one is late because the truth arrives late.
- **A rule for when to throw it out.** A retraining trigger, decided in advance, so the decision isn't made by whoever shouts loudest.
- **A way to put the old one back.** Rollback, practiced before you need it.

The rest of this chapter is those four parts, on Riverstone's defect model, with real tools and measured results.

---

## 56.1 What breaks after launch

---

```
import numpy as np

stream = np.load("production_data/stream.npz", allow_pickle=True)
features, labels, week = stream["features"], stream["labels"], stream["week"]
brightness, kind = stream["brightness"], stream["kind"]

print(f"{len(labels):,} parts over {week.max() + 1} weeks, {labels.sum():,} defective")
counts = dict(zip(*np.unique(kind, return_counts=True)))
print("defect types seen:", {str(k): int(v) for k, v in counts.items()})
print(f"mean brightness, week 0: {brightness[week == 0].mean():.3f}")
print(f"mean brightness, week 23: {brightness[week == 23].mean():.3f}")
print(f"the new 'flash' defect first appears in week {week[kind == 'flash'].min()}")
```

```
12,000 parts over 24 weeks, 1,053 defective
defect types seen: {'flash': 69, 'good': 10947, 'scratch': 347, 'short_shot': 320, 'void': 317}
mean brightness, week 0: 0.392
mean brightness, week 23: 0.448
the new 'flash' defect first appears in week 16
```

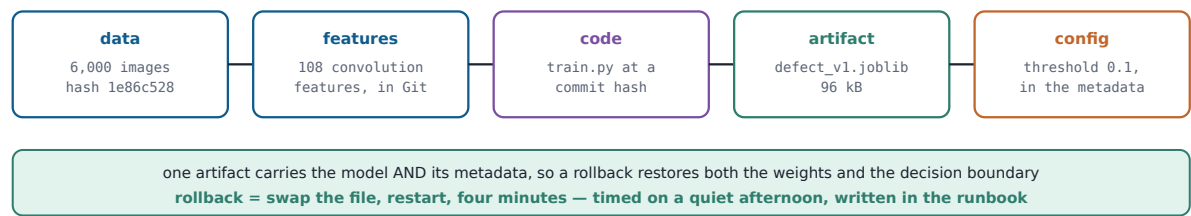
**Line by line:** `np.load(..., allow_pickle=True)` reads the compressed archive the simulator wrote; `allow_pickle` is needed only because one array holds strings. `np.unique(..., return_counts=True)` returns the distinct values and how often each occurs, which `dict(zip(*...))` turns into a readable tally.

Two changes are hidden in that data, and they are the two that matter in practice:

| Change                                                | Name                                            | When        | What a monitor sees                              |
|-------------------------------------------------------|-------------------------------------------------|-------------|--------------------------------------------------|
| The lamps are replaced, so the belt looks brighter    | <b>data drift</b> (the inputs change)           | weeks 8–15  | a large, obvious shift in the input distribution |
| A new mould produces "flash", a defect nobody labeled | <b>concept drift</b> (the relationship changes) | weeks 16–23 | nothing, until labels arrive                     |

Section 56.8 measures both, and the result is not what most articles about drift will tell you.

The five things that must be versioned, and where they travel



If any one of the five is missing, you cannot reproduce the model, compare it with its successor, or explain a decision six months later.

Figure 56.1 — "The model" is five things, and a rollback has to restore all of them.

## 56.2 Versioning: "the model" is five things

| What                  | Why it must be versioned                                        | How                                                                      |
|-----------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Data</b>           | "trained on last March's images" is not reproducible            | a snapshot, a hash, or a query pinned to a date (Chapter 32's snapshots) |
| <b>Features</b>       | the same raw image gives different features if the code changed | the feature code in Git, its version recorded with the model             |
| <b>Code</b>           | the training script is part of the model                        | Git commit hash, recorded in the run                                     |
| <b>Model artifact</b> | the trained weights themselves                                  | a file in a registry, with a version number                              |
| <b>Config</b>         | the threshold is a business decision, and it moves              | in the artifact's metadata, not hard-coded in the service                |

A **model registry** is the index of those artifacts: name, version, stage (staging, production, archived), the run that produced it, and its metrics. MLflow's registry, SageMaker's, Vertex's, and a folder with a strict naming convention are all implementations of the same idea, and the folder is a legitimate starting point for one model.

**Watch out: the threshold is part of the model.** Chapter 53 chose 0.01 from the cost of a miss against a false alarm. If that number lives in the service's code and the model artifact lives in a registry, you have two things that must be changed together and are stored apart. Put the threshold in the artifact's metadata, as `serve.py` does, and a rollback returns both.

## 56.3 Experiment tracking

Four training runs, with their parameters, metrics, and the model itself recorded in a local MLflow store:

```
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")    # one local file: no server, no account, real
tracking
mlflow.set_experiment("riverstone-defect")

def run_one(hidden, max_iter, threshold):
    with mlflow.start_run(run_name=f"mlp-{hidden}-t{threshold}"):
        model = MLPClassifier(hidden_layer_sizes=(hidden,), max_iter=max_iter, random_state=53)
        model.fit(X_train, y_train)
        probabilities = model.predict_proba(X_test)[: , 1]
        tn, fp, fn, tp = confusion_matrix(y_test, probabilities > threshold).ravel()
        metrics = {"recall": tp / (tp + fn), "precision": tp / (tp + fp) if tp + fp else 0.0,
                  "false_alarms": float(fp), "missed": float(fn),
                  "cost_rupees": float(fn * 4000 + fp * 40)}    # Chapter 53's business metric
        mlflow.log_params({"hidden_units": hidden, "max_iter": max_iter, "threshold": threshold,
                          "features": X.shape[1], "training_rows": len(X_train)})
        mlflow.log_metrics(metrics)
        mlflow.sklearn.log_model(model, name="model", skops_trusted_types=[...])
        return {"hidden": hidden, "threshold": threshold, **metrics}
```

### Line by line:

- `set_tracking_uri("sqlite:///mlflow.db")` puts the whole store in one file next to the code. (MLflow 3 deprecated the older plain-folder store; a local SQLite file is the modern equivalent of "no infrastructure".)
- `set_experiment` groups runs, so a year of work is browsable rather than a heap.
- `with mlflow.start_run(...)` opens a run and closes it even if training raises, which is what you want: a failed run that recorded its parameters is still evidence.
- `log_params` records what you chose; `log_metrics` records what happened. **Log the business metric**, not only the statistical one: `cost_rupees` is what the plant manager compares, and it is the column you will sort by.
- `log_model` stores the artifact itself, so the run is not just a note about a model that has since been overwritten.

Running the four settings from `train.py`:

```
# terminal, in companion/ch56
$ python3 train.py
```

| hidden | threshold | recall | precision | false alarms | cost (Rs) |
|--------|-----------|--------|-----------|--------------|-----------|
| 24     | 0.1       | 0.933  | 0.902     | 12           | 32,480    |
| 48     | 0.1       | 0.950  | 0.950     | 6            | 24,240    |
| 48     | 0.05      | 0.958  | 0.884     | 15           | 20,600    |
| 96     | 0.1       | 0.933  | 0.982     | 2            | 32,080    |

And querying them afterwards, which is the entire point:

```
import warnings
import mlflow

warnings.filterwarnings("ignore")
mlflow.set_tracking_uri("sqlite:///mlflow.db")
runs = mlflow.search_runs(experiment_names=["riverstone-defect"], order_by=["metrics.cost_rupees ASC"])
columns = ["tags.mlflow.runName", "params.hidden_units", "params.threshold",
           "metrics.recall", "metrics.cost_rupees"]
print(runs[columns].to_string(index=False))
print(f"\ncheapest run: {runs.iloc[0]['tags.mlflow.runName']}")
```

| tags.mlflow.runName | params.hidden_units | params.threshold | metrics.recall | metrics.cost_rupees |
|---------------------|---------------------|------------------|----------------|---------------------|
| mlp-48-t0.05        | 48                  | 0.05             | 0.957983       | 20600.0             |
| mlp-48-t0.1         | 48                  | 0.1              | 0.949580       | 24240.0             |
| mlp-96-t0.1         | 96                  | 0.1              | 0.932773       | 32080.0             |
| mlp-24-t0.1         | 24                  | 0.1              | 0.932773       | 32480.0             |

```
cheapest run: mlp-48-t0.05
```

**Read the ordering.** The cheapest configuration is not the most accurate by precision, nor the biggest network. The 96-unit model has the best precision (0.982) and one of the worst costs, because it misses more defects. **Sorting by the business metric puts the right run first**, and that is why the metric belongs in the tracking store.

## 56.4 Packaging: what travels with the model

```
import joblib

bundle = joblib.load("models/defect_v1.joblib")
model, metadata = bundle["model"], bundle["metadata"]

for key, value in metadata.items():
    print(f" {key:<15} {value}")
print(f"\nthe artifact is {round(__import__('os').path.getsize('models/defect_v1.joblib') / 1024)} kB")
```

```
version      defect-v1
trained_on   ch53 defect_data (6,000 images, seed 53)
features     108
threshold    0.1
sklearn      1.8.0
trained_at   2026-03-02T09:00:00
training_rows 6000
data_hash    1e86c528a4bfe346
metrics      {'recall': 0.95, 'precision': 0.95}
```

the artifact is 96 kB

**Line by line:** the file holds a dictionary with two keys, the model and its metadata, so **they cannot be separated**. The metadata answers the questions somebody will ask in six months: what was it trained on, how many rows, which library version, what threshold, and what it scored when it was built. `data_hash` is the cheap version of data versioning: a hash of the training features, which proves whether two runs used the same data.

**On pickles.** `joblib` and `pickle` store Python objects by storing instructions for rebuilding them, which means **loading a pickle can run arbitrary code**. That is fine for a file your own pipeline wrote; it is a serious vulnerability for a file that arrived from outside. MLflow 3 makes the point for you: it refuses to save a scikit-learn model containing types it doesn't recognize until you list them as trusted, which is why `train.py` names `AdamOptimizer` explicitly. Treat a model artifact like an executable: signed, stored where you control access, and never loaded from an untrusted source.

## 56.5 Serving

Three shapes, and most teams need only the first two:

| Pattern          | The model runs                        | Latency          | Fits                                                     |
|------------------|---------------------------------------|------------------|----------------------------------------------------------|
| <b>Batch</b>     | on a schedule, over many rows at once | minutes to hours | scoring yesterday's parts, churn lists, demand forecasts |
| <b>Online</b>    | when a request arrives                | milliseconds     | a camera on a line, a fraud check, a recommendation      |
| <b>Streaming</b> | as events arrive on a queue           | seconds          | continuous sensors, clickstreams                         |

Riverstone's camera needs online serving. `serve.py` is the whole service:

```

class Part(BaseModel):
    part_id: str = Field(min_length=1, max_length=40)
    features: list[float] = Field(min_length=108, max_length=108)

@app.get("/health")
def health() -> dict:
    """Liveness and readiness in one: the service is up AND the model is loaded."""
    return {"status": "ok", "model_version": metadata["version"], "threshold":
metadata["threshold"]}

@app.post("/predict")
def predict(part: Part) -> dict:
    started = time.perf_counter()
    values = np.asarray(part.features, dtype=float)
    if not np.isfinite(values).all():
        raise HTTPException(status_code=422, detail="features contain NaN or infinity")
    probability = float(model.predict_proba(values.reshape(1, -1))[0, 1])
    verdict = "defective" if probability > metadata["threshold"] else "good"
    elapsed_ms = (time.perf_counter() - started) * 1000
    log.info(json.dumps({"part_id": part.part_id, "probability": round(probability, 4),
                        "verdict": verdict, "model_version": metadata["version"],
                        "latency_ms": round(elapsed_ms, 2), "mean_feature":
round(float(values.mean()), 4)}))
    return {"part_id": part.part_id, "probability": round(probability, 4), "verdict": verdict,
            "model_version": metadata["version"], "threshold": metadata["threshold"],
            "latency_ms": round(elapsed_ms, 2)}

```

### Line by line, for the parts that are about production rather than Python:

- **Part** is a Pydantic model, and those two `Field` constraints are the service's input contract. A request with 50 features is rejected with a 422 before the model sees it, which is the difference between a clear error and a wrong prediction on garbage.
- `/health` returns the model version, not just "ok". A health check that passes while the wrong model is loaded has told you nothing.
- `np.isfinite(values).all()` catches NaN and infinity, which arrive from real cameras more often than anyone expects and which scikit-learn will happily propagate.
- **The threshold comes from `metadata`**, so the deployed decision boundary is whatever the artifact says it is (section 56.2).
- **The log line is JSON**, with one field per thing monitoring needs: the version, the probability, the verdict, the latency, and a cheap summary of the input (`mean_feature`). Section 56.7 builds the monitors on exactly these fields. Log structured data from day one; nobody ever regrets it.

Exercising it in-process, which is also how you test a service in CI (Chapter 32):

```

import logging
import numpy as np
from fastapi.testclient import TestClient

import sys
sys.path.insert(0, ".")
logging.getLogger("defect-service").setLevel(logging.WARNING) # quiet the per-request log here
from serve import app

client = TestClient(app)
print(client.get("/health").json())

defective = int(np.where((week == 0) & (labels == 1))[0][0])
good = int(np.where((week == 0) & (labels == 0))[0][0])
for name, index in [("a defective part", defective), ("a good part", good)]:
    response = client.post("/predict", json={"part_id": f"P-{index:05d}",
                                             "features": features[index].tolist()})

    body = response.json()
    print(f"{name:<18} -> {body['verdict']:<10} probability {body['probability']:.4f} "
          f"(model {body['model_version']})")

bad = client.post("/predict", json={"part_id": "P-00003", "features": [0.1] * 50})
print(f"a 50-feature request -> HTTP {bad.status_code}: {bad.json()['detail'][0]['msg']}")

```

```

{'status': 'ok', 'model_version': 'defect-v1', 'threshold': 0.1}
a defective part -> defective probability 1.0000 (model defect-v1)
a good part -> good probability 0.0000 (model defect-v1)
a 50-feature request -> HTTP 422: List should have at least 108 items after validation, not 50

```

Measured on the author's machine, the model itself takes about 0.2 ms per part and the whole HTTP round trip about 1.8 ms at the median, 2.1 ms at the 95th percentile. That is far inside the camera's budget, and the useful lesson is the ratio: **the model is a small fraction of the latency**, and most optimization effort in serving goes into everything around it.

## 56.6 Deployment patterns

| Pattern           | What happens                                                                             | Buys you                             | Costs                                           |
|-------------------|------------------------------------------------------------------------------------------|--------------------------------------|-------------------------------------------------|
| <b>Big bang</b>   | replace the old model with the new one                                                   | nothing                              | everything, when it's wrong                     |
| <b>Shadow</b>     | the new model scores every request beside the old one; only the old one's answer is used | real production data, zero risk      | double compute; no feedback from real decisions |
| <b>Canary</b>     | 5% of traffic uses the new model, then 25%, then all                                     | limited blast radius, real decisions | needs traffic splitting and per-version metrics |
| <b>Blue-green</b> | two complete environments, traffic switched at once, old one kept warm                   | instant rollback                     | two environments to pay for                     |
| <b>A/B test</b>   | versions compared as an experiment (Chapter 30)                                          | a measured causal answer             | needs enough volume and a real metric           |

For a defect model on one line, the sequence that works is **shadow, then canary, then keep the previous artifact one click away**. Shadow mode is especially valuable here because the model's ground truth arrives late: run the new version silently for two weeks, collect its disagreements with

the current one, and have the QC inspector adjudicate *only the disagreements*, which is a few dozen parts rather than thousands.

```
import numpy as np
import joblib
from sklearn.neural_network import MLPClassifier

current = joblib.load("models/defect_v1.joblib")["model"]
recent = np.isin(week, range(16, 20))
candidate = MLPClassifier(hidden_layer_sizes=(48,), max_iter=300, random_state=56).fit(
    np.vstack([np.load("../ch53/defect_data/features.npy"), features[recent]]),
    np.concatenate([np.load("../ch53/defect_data/labels.npy"), labels[recent]]))

shadow_weeks = np.isin(week, range(20, 24))
current_says = current.predict_proba(features[shadow_weeks])[:, 1] > 0.1
candidate_says = candidate.predict_proba(features[shadow_weeks])[:, 1] > 0.1
disagreements = current_says != candidate_says
truth = labels[shadow_weeks]

print(f"parts scored in shadow: {shadow_weeks.sum():,}")
print(f"the two models disagree on {disagreements.sum()} of them ({disagreements.mean():.1%}")
print(f"of those disagreements, the candidate is right {int((candidate_says == truth)
[disagreements].sum())} times "
      f"and the current model is right {int((current_says == truth)[disagreements].sum())} times")
```

```
parts scored in shadow: 2,000
the two models disagree on 33 of them (1.7%)
of those disagreements, the candidate is right 17 times and the current model is right 16 times
```

**Line by line:** the candidate is trained on the original data plus four recent weeks; `shadow_weeks` are the four weeks after that, which the candidate has never seen. The interesting quantity is not either model's accuracy, it is **where they disagree**, because that is the only set a human needs to look at. In production, those part ids go to the inspector, and their verdicts become both the decision and next month's training data.

**And look at how close the verdict is:** on the 33 parts where they disagree, the candidate is right 17 times and the current model 16. That is a coin flip, not a case for shipping. Two weeks of shadow traffic has told Riverstone that the candidate is *not clearly better where it matters*, which is exactly what shadow mode is for and exactly the result that a headline accuracy comparison (section 56.9) would have hidden.

---

## 56.7 Monitoring, in three layers

---

| Layer | Question | Signal | Delay |
|-------|----------|--------|-------|
|-------|----------|--------|-------|

### Three layers of monitoring, and what each one can see

|                                                   |                                                                          |               |
|---------------------------------------------------|--------------------------------------------------------------------------|---------------|
| <b>service</b><br>is it up and fast?              | uptime, error rate, p95 latency, memory<br>delay: seconds                | page a human  |
| <b>input</b><br>does the data look like training? | feature distributions, PSI, KS, missing rates<br>delay: minutes          | daily digest  |
| <b>output</b><br>what is it saying?               | predicted-defective rate per shift vs the QC tally<br>delay: hours       | daily digest  |
| <b>outcome</b><br>is it still right?              | recall from a 2% audit of parts the model passed<br>delay: days to weeks | weekly review |

The flash defect was invisible to the first three layers and obvious to the fourth, which is the slowest and the only one that measures whether the model is still right. Ground truth is late; plan for it rather than hoping.

Figure 56.2 – Each layer sees a different failure, and the slowest one is the only one that measures correctness.

| **Service** | is it up and fast? | uptime, error rate, latency percentiles, memory | seconds | | **Input** | does the data look like the training data? | feature distributions, missing rates, ranges, drift statistics | minutes | | **Output and outcome** | is it still right? | prediction distribution, and accuracy once labels arrive | hours to weeks |

The first layer is ordinary software monitoring. The second is section 56.8. The third has a property that makes it hard and that most ML monitoring articles skate over: **ground truth is late**. A part flagged defective is checked within the hour, but a part passed as good is only revealed as wrong when a customer complains, weeks later. So the third layer has two streams:

- **Fast proxy**: the share of parts predicted defective, per shift. It needs no labels, and a sudden move in it is always worth a look.
- **Slow truth**: a sampled audit. Riverstone re-inspects 2% of passed parts, which is the only way a missed-defect rate is ever measured.

```
print(f'{"week":>5}{{"parts":>7}{{"predicted defective":>21}{{"actually defective":>20}}}')
model = joblib.load("models/defect_v1.joblib")["model"]
for w in (0, 7, 12, 16, 20, 23):
    rows = week == w
    predicted = (model.predict_proba(features[rows])[:, 1] > 0.1).mean()
    print(f'{"w":>5}{rows.sum():>7}{predicted:>20.1%}{labels[rows].mean():>19.1%}')
```

| week | parts | predicted defective | actually defective |
|------|-------|---------------------|--------------------|
| 0    | 500   | 8.2%                | 8.0%               |
| 7    | 500   | 7.4%                | 7.2%               |
| 12   | 500   | 7.4%                | 7.2%               |
| 16   | 500   | 7.8%                | 9.2%               |
| 20   | 500   | 10.4%               | 11.8%              |
| 23   | 500   | 8.2%                | 9.0%               |

**Read the two columns together.** In weeks 0 to 12 they track each other. From week 16 the *actual* defect rate rises (the new mould) while the *predicted* rate does not, because the model cannot see the new defect. **A gap opening between what the model says and what the line produces is the cheapest early warning there is**, and it needs only a daily count and a QC tally.

**Watch out: alert on what you will act on.** A dashboard nobody looks at is not monitoring, and an alert that fires weekly gets muted in a fortnight. Riverstone's rule: page a human only for the service layer (down, or error rate above 1%); everything else is a daily digest with a named owner. Chapter 47's data-quality alerting makes the same argument for pipelines.

## 56.8 Drift, measured

Two standard statistics, computed by hand before reaching for a library.

**Population stability index (PSI)** compares a recent sample with a reference sample, bucket by bucket:

```
def psi(reference, recent, buckets=10):
    """How far has this distribution moved? Rules of thumb: <0.1 stable, 0.1-0.25 watch, >0.25
    investigate."""
    cuts = np.quantile(reference, np.linspace(0, 1, buckets + 1))
    cuts[0], cuts[-1] = -np.inf, np.inf
    reference_share = np.clip(np.histogram(reference, cuts)[0] / len(reference), 1e-4, None)
    recent_share = np.clip(np.histogram(recent, cuts)[0] / len(recent), 1e-4, None)
    return float(((recent_share - reference_share) * np.log(recent_share / reference_share)).sum())

baseline = brightness[week < 8]
print(f"{'week':>5}{ 'PSI':>9}{ 'KS':>8}   verdict")
from scipy import stats
for w in (3, 8, 10, 13, 16, 20, 23):
    recent = brightness[week == w]
    value = psi(baseline, recent)
    ks = stats.ks_2samp(baseline, recent).statistic
    verdict = "stable" if value < 0.1 else "watch" if value < 0.25 else "investigate"
    print(f"{'w':>5}{value:>9.3f}{ks:>8.3f}   {verdict}")
```

| week | PSI   | KS    | verdict     |
|------|-------|-------|-------------|
| 3    | 0.007 | 0.042 | stable      |
| 8    | 0.065 | 0.116 | stable      |
| 10   | 0.614 | 0.270 | investigate |
| 13   | 2.959 | 0.562 | investigate |
| 16   | 4.107 | 0.689 | investigate |
| 20   | 3.981 | 0.704 | investigate |
| 23   | 4.246 | 0.680 | investigate |

**Line by line:**

- `np.quantile(reference, ...)` cuts the *reference* period into ten equal-sized buckets. Using the reference's own quantiles is the point: the buckets are defined by the world the model was trained on.
- `cuts[0], cuts[-1] = -np.inf, np.inf` opens the ends, so a value beyond anything seen before still lands somewhere.
- `np.clip(..., 1e-4, None)` prevents an empty bucket producing a division by zero or an infinite logarithm, which is the standard fudge and worth knowing about.
- The sum is over buckets of  $(\text{recent share} - \text{reference share}) \times \log(\text{recent share} / \text{reference share})$ : zero when the distributions match, growing as they separate.

- **The KS statistic** (`ks_2samp`) is the largest gap between the two cumulative distributions: 0 means identical, 1 means no overlap. It needs no bucketing and is a good cross-check.
- The thresholds (0.1 and 0.25) are conventions from credit scoring, not laws. Calibrate them on your own stable period: run PSI on week 3 against weeks 0 to 7, see what "normal" looks like, and set the threshold above it.

## The finding that matters

Now put drift next to the thing everyone assumes it predicts:

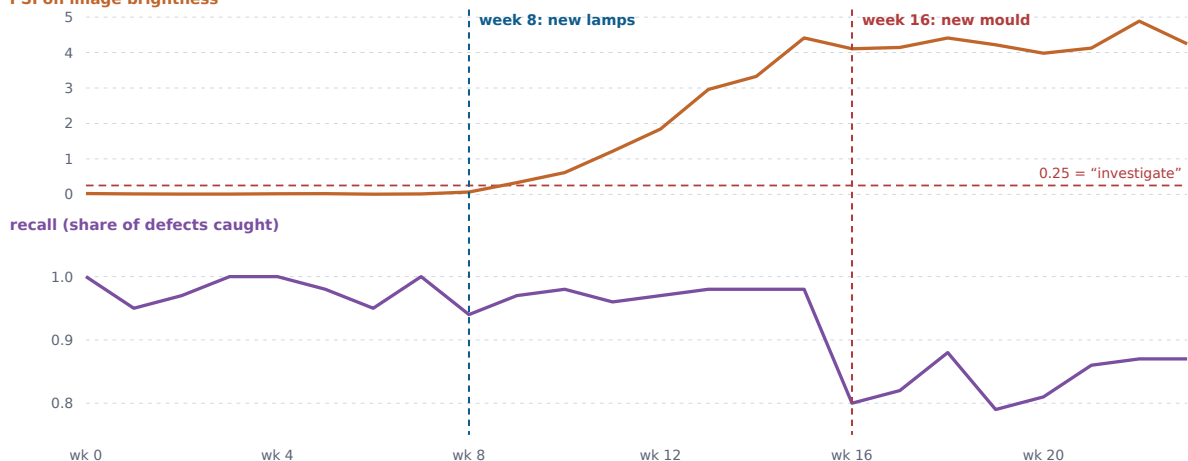
```
from sklearn.metrics import confusion_matrix

print(f"{'week':>5}{{'PSI':>8}{{'recall':>9}{{'false alarms':>14}  what is happening")
notes = {0: "baseline", 7: "baseline", 9: "new lamps, week 2", 13: "new lamps, week 6",
        15: "new lamps, week 8", 17: "new mould, week 2", 20: "new mould, week 5", 23: "new mould, week 8"}
for w in (0, 7, 9, 13, 15, 17, 20, 23):
    rows = week == w
    predicted = model.predict_proba(features[rows])[:, 1] > 0.1
    tn, fp, fn, tp = confusion_matrix(labels[rows], predicted, labels=[0, 1]).ravel()
    print(f"{'w':>5}{psi(baseline, brightness[rows]):>8.2f}{tp / (tp + fn):>8.2f}{fp:>14}
{notes[w]}")
```

| week | PSI  | recall | false alarms | what is happening |
|------|------|--------|--------------|-------------------|
| 0    | 0.02 | 1.00   | 1            | baseline          |
| 7    | 0.01 | 1.00   | 1            | baseline          |
| 9    | 0.33 | 0.97   | 1            | new lamps, week 2 |
| 13   | 2.96 | 0.98   | 2            | new lamps, week 6 |
| 15   | 4.41 | 0.98   | 2            | new lamps, week 8 |
| 17   | 4.14 | 0.82   | 3            | new mould, week 2 |
| 20   | 3.98 | 0.81   | 4            | new mould, week 5 |
| 23   | 4.25 | 0.87   | 2            | new mould, week 8 |

### Input drift is not model decay: six months on Riverstone's line

#### PSI on image brightness



Weeks 8-15: PSI reaches 4.4, seventeen times the "investigate" threshold, and recall never drops below 0.94.

**Weeks 16-23: PSI is flat and recall falls fifteen points. The alarm that cried wolf in May stayed silent in July.**

Figure 56.3 – The drift alarm fired when nothing was wrong, and stayed silent when something was.

**This table is the most useful thing in the chapter, and it says the opposite of the standard advice.**

- **Weeks 8 to 15: the drift alarm screams and the model is fine.** PSI reaches 4.4, seventeen times the "investigate" threshold, while recall never drops below 0.94. The inputs changed enormously; the features the model relies on did not change in a way that mattered.
- **Weeks 16 to 23: the model breaks and the drift alarm says nothing new.** Recall falls by around 15 points, and PSI is flat, because the incoming images look exactly like last month's. The failure is in the *relationship* between input and label, and no amount of input monitoring can see it.

The conclusion is not that drift statistics are useless: they are cheap, immediate, and they told the truth both times (the inputs *had* changed; then they *hadn't*). The conclusion is that **input drift is not model decay**, and a monitoring plan built only on drift will page you for the wrong thing in April and stay silent in June. You need both layers, and you need the sampled labels.

## 56.9 Retraining

Triggers, in the order teams should adopt them:

| Trigger            | When it fires                                      | Comment                                                                  |
|--------------------|----------------------------------------------------|--------------------------------------------------------------------------|
| <b>Scheduled</b>   | monthly, quarterly                                 | the simplest thing that works, and the right default                     |
| <b>Performance</b> | measured accuracy falls below a floor              | needs labels, so it fires late                                           |
| <b>Drift</b>       | PSI above a threshold for $n$ consecutive days     | fires early, and sometimes for nothing (see above)                       |
| <b>Event</b>       | a known change: new mould, new camera, new product | the best trigger there is, and it comes from the business, not a monitor |
| <b>Data volume</b> | $n$ new labeled examples available                 | good where labels accumulate naturally                                   |

Riverstone's rule, written down: **retrain monthly, and immediately on any event the plant tells us about.** The new mould in week 16 is exactly such an event, and the retraining that follows should have been triggered by the production meeting, not by the model's recall falling.

```
from sklearn.metrics import confusion_matrix

def score(candidate, weeks, threshold=0.1):
    rows = np.isin(week, weeks)
    predicted = candidate.predict_proba(features[rows])[:, 1] > threshold
    tn, fp, fn, tp = confusion_matrix(labels[rows], predicted, labels=[0, 1]).ravel()
    return tp / (tp + fn), fp

for name, candidate_model in [("current (defect-v1)", model), ("retrained (defect-v2)",
candidate)]:
    recall_new, alarms_new = score(candidate_model, range(20, 24))
    recall_old, alarms_old = score(candidate_model, range(0, 8))
    print(f"{name:<22} recent weeks: recall {recall_new:.2f}, false alarms {alarms_new:>3}"
          f"      | old weeks (regression check): recall {recall_old:.2f}, false alarms
{alarms_old:>3}")
```

```
current (defect-v1)    recent weeks: recall 0.85, false alarms 10 | old weeks (regression
check): recall 0.98, false alarms 10
retrained (defect-v2) recent weeks: recall 0.91, false alarms 20 | old weeks (regression
check): recall 0.99, false alarms 30
```

**Read it as the shipping decision it is.** The retrain does what it was meant to: recall on the recent weeks rises from 0.85 to 0.91, because the model has now seen the flash defect. It also **doubles the false alarms on the recent weeks and triples them on the old ones**, and the regression check confirms it hasn't forgotten the original defects (0.99).

Is that shippable? Chapter 53's arithmetic answers it: at ₹4,000 a miss and ₹40 a false alarm, the retrain saves more than it costs. But the threshold that was optimal for `defect-v1` is not optimal for `defect-v2`, and **re-tuning the threshold is part of retraining**, not an afterthought. A retrain that ships with the old threshold is a common and avoidable way to make a model worse.

**The guardrails on any automated retrain:**

- **A held-out set the retrain never sees**, including old data, so forgetting is caught.
- **A floor**: the new model must beat the old one on the business metric, or it does not ship.
- **A regression check** on the original data, as above.
- **A human approval** for the first several retrains, until the process has earned trust.
- **The previous artifact retained**, and a rollback you have actually practiced.

**Watch out: retraining on the model's own output is how a model eats itself.** If the training labels come from parts the model flagged, it only ever learns about defects it already catches, and its blind spots become permanent. Riverstone's 2% audit of *passed* parts exists exactly to break that loop, and it is the single most valuable data collection in the whole system.

## 56.10 When it goes wrong

**The runbook**, which should be one page pinned next to the dashboard:

| Step           | Action                                                                                 |
|----------------|----------------------------------------------------------------------------------------|
| 1. Confirm     | Is it the model, the data, or the service? Check the three layers in order             |
| 2. Contain     | Roll back to the previous model version, or switch the line to manual inspection       |
| 3. Communicate | Tell the plant and QC what is happening and what they should do meanwhile              |
| 4. Diagnose    | Logs first: model version, input summaries, prediction distribution, per-shift metrics |
| 5. Fix         | The smallest change that restores service. Retraining is rarely the fastest fix        |
| 6. Learn       | Post-mortem, blameless, with an action that would have caught it sooner                |

**Rollback is the step people skip practising.** With the artifact-plus-metadata packaging of section 56.4, it is a file swap and a restart, and the threshold comes back with it. Time it once, on a quiet afternoon, and write the number in the runbook: *"rollback takes four minutes"* is a sentence that

changes how a plant manager feels about the whole project.

**The post-mortem** that matters is not "the model degraded". It is the sentence that starts "*we would have caught this sooner if...*", and its answer becomes the next monitor. For the flash defect, that sentence is: "*if the production meeting's note about the new mould had reached the ML on-call rota.*" The fix is a process change, not a model change, and it is typical.

## 56.11 How far up the ladder to climb

| Level                          | What it looks like                                                                                   | Who should be here                                   |
|--------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>0. Notebook</b>             | a model in someone's notebook, run by hand                                                           | proofs of concept only                               |
| <b>1. Reproducible</b>         | training is a script in Git, artifacts versioned with metadata, runs tracked                         | <b>every company with one model in production</b>    |
| <b>2. Served and monitored</b> | an endpoint or a scheduled batch, three-layer monitoring, a runbook, practiced rollback              | <b>Riverstone, and most mid-sized companies</b>      |
| <b>3. Automated pipelines</b>  | CI/CD for models, automated retraining with guardrails, a registry with stages                       | companies with several models and a team to run them |
| <b>4. Platform</b>             | feature store, shared serving infrastructure, self-serve deployment, lineage across dozens of models | large organizations with many teams                  |

**Most companies should stop at level 2, and most articles are written from level 4.** A feature store is excellent when twenty models share features and terrible when one model has 108 of them.

Kubernetes is excellent at scale and an enormous tax on one service serving one camera. The honest question is: *what would break if we did this by hand once a month?* If the answer is "nothing", do it by hand and spend the time on the audit sample instead.

What does not scale down, and is worth doing at every level: **version the artifact with its metadata, track the runs, log structured predictions, monitor in three layers, and be able to roll back.** None of that needs a platform. All of it can be done in a week.

## Common mistakes and how to spot them

| Mistake                                                     | Symptom                                                     | Fix                                            |
|-------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------|
| The threshold lives in the service, the model in a registry | A rollback restores the model and not the decision boundary | Threshold in the artifact metadata             |
| No data version recorded                                    | "Which data was this trained on?" has no answer             | A hash or a pinned snapshot in the metadata    |
| Metrics tracked in a spreadsheet                            | Twelve runs nobody can compare                              | An experiment tracker, even a local SQLite one |
| Only statistical metrics logged                             | The best run by F1 is the worst by cost                     | Log the business metric and sort by it         |
| Health check returns "ok" only                              | Passes while the wrong model is loaded                      | Return the model version too                   |

| Mistake                                   | Symptom                                                                     | Fix                                                                             |
|-------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Unvalidated inputs                        | A wrong prediction on garbage instead of a clear 422                        | A schema on the request, and a finite check                                     |
| Unstructured logs                         | Monitoring you cannot build                                                 | One JSON line per prediction, with version, probability, latency, input summary |
| Drift monitoring alone                    | Pages you for harmless input changes, silent when the model actually breaks | Three layers, including sampled ground truth                                    |
| Treating PSI thresholds as laws           | Alert fatigue within a month                                                | Calibrate on your own stable period                                             |
| No sampled audit of passed items          | Missed defects are invisible until a customer finds them                    | A fixed percentage of passed parts re-checked                                   |
| Retraining on the model's own flags       | Blind spots become permanent                                                | Audit sample of passed parts feeds training                                     |
| Shipping a retrain with the old threshold | Recall up, cost up, nobody understands why                                  | Re-tune the threshold as part of retraining                                     |
| No held-out regression check              | The new model forgets the old defects                                       | Evaluate on old data as well as recent                                          |
| Rollback never practiced                  | A four-minute fix takes two hours during an incident                        | Time it on a quiet afternoon; write the number down                             |
| Building a platform for one model         | Six months of infrastructure, one endpoint                                  | Level 2 is the right destination for most companies                             |

## In the real world: the fortnight the line got worse and nobody knew

Riverstone's defect camera runs quietly from March. In August, a customer returns a pallet of lids with a thin tail of plastic along one edge: flash, from the new mould that started in week 16 of the log.

The post-mortem is uncomfortable, because every individual decision was defensible.

- **The drift monitor had been amber since May**, when the plant replaced the overhead lamps. PSI went from 0.01 to over 4, the daily digest said "investigate" for six weeks, and the model was fine the whole time. By July, everyone had learned that the drift alert meant nothing.
- **When the new mould arrived in July, the drift number did not move**, because the pictures looked the same. The alert that had been crying wolf stayed silent for the real event.
- **Recall fell from 0.98 to 0.81** and nobody could see it, because nobody was checking parts the model had passed.
- **The production meeting knew about the new mould on the day it was installed.** That note never reached the person running the model.

The fixes are all cheap, and none of them is a better model:

1. **A 2% audit of passed parts**, which makes recall measurable within a week instead of a customer complaint.

2. **The predicted-defective rate per shift on the same chart as QC's own tally.** The gap between the two lines opens in week 16 and is visible to anyone glancing at it.
3. **The event trigger:** the production meeting's change log now goes to whoever owns the model, and any new mould, resin, or camera triggers a review.
4. **Drift thresholds recalibrated** on Riverstone's own stable period, so amber means something.
5. **A retrain including flash examples**, shipped after shadow mode, with the threshold re-tuned and the old artifact kept one click away.

**What Meera writes in the post-mortem:** *"The model didn't fail; our monitoring measured the wrong thing. Input drift told us the lamps changed, which didn't matter, and told us nothing when the parts changed, which did. We are adding a 2% audit of passed parts and a line in the production meeting's checklist. The model change is the smallest part of this fix."*

The pattern is worth carrying into any ML project: **the failures are rarely in the model, and the fixes are rarely modeling.**

---

## Tools

---

Versions used, checked in September 2026:

- **Python 3.12.3, scikit-learn 1.8.0, NumPy, SciPy, joblib.**
- **FastAPI 0.141.1** and **uvicorn** for serving; `fastapi.testclient` for testing the service in CI (needs `httpx2` installed).
- **MLflow 3.16.1** for tracking and the model registry, with a local SQLite backend. Note that MLflow 3 deprecated the plain-folder store and requires trusted types to be declared when logging scikit-learn models.
- **Alternatives worth knowing:** Weights & Biases (tracking), DVC (data versioning), BentoML and Seldon (serving), Evidently and NannyML (drift and performance monitoring), SageMaker / Vertex AI / Azure ML (managed end to end), Airflow or Dagster for the scheduled parts (Chapter 46).
- **Containers and orchestration** (Docker, Kubernetes) are how this runs at scale. They change the packaging and deployment steps, not the discipline.
- **Companion files** in `ch56/`: `simulate_production.py` (the 24-week stream, seed 56), `train.py` (tracked training runs), `serve.py` (the FastAPI service), `models/defect_v1.joblib`, and `ch56_check.py`.

**Simplification note.** This is a one-machine simulation of production: no cloud, no container, no real traffic, and a "line" that is a NumPy array. The code, the metrics, and the failures are real; the scale is not. What changes at real scale: the serving layer gets containers and a load balancer, the store gets a database, monitoring gets a time-series system, and the audit sample gets a workflow tool. What does not change: any of section 56.11's level-2 practices.

## The project: run a model for six months in an afternoon

---

**Goal:** take a model you have trained and put it through a full production lifecycle, with every decision written down.

**Option A: your own model.** Any model you built in Chapters 38 to 53, with a stream of data you can replay in time order.

**Option B: Riverstone.** Use this chapter's stream.

### Steps:

1. **Package it** with metadata: version, data hash, training rows, library versions, threshold, metrics.
2. **Track three training runs** and record the business metric alongside the statistical ones. Sort by the business metric.
3. **Serve it** behind an endpoint with input validation, a health check that returns the version, and one structured log line per prediction.
4. **Write the three-layer monitoring plan** before you look at any results: what you measure, how often, who is told, and what they do.
5. **Replay the stream week by week**, computing your monitors as you go. Record when each one first fires.
6. **Find the two events** in the data. Which monitor caught which, and how late?
7. **Retrain**, with a held-out regression check, and re-tune the threshold. Decide, in writing, whether it ships.
8. **Practise the rollback** and time it.
9. **Write the post-mortem** for the failure you found, including the sentence "we would have caught this sooner if..."

**Deliverables:** the packaged artifact, the tracked runs, the service, the monitoring plan, the week-by-week table, the retraining decision, and the post-mortem.

### Stretch goals:

- Add shadow mode and adjudicate only the disagreements.
- Add a scheduled batch scorer alongside the endpoint, and compare cost per thousand predictions.
- Compute drift per feature rather than on one summary, and see which features move.
- Add a CI job (Chapter 32) that fails if the packaged model's metrics fall below the floor.
- Estimate what this would cost per month on a cloud provider, at 5,000 parts a day.

---

## You've got it when...

- ☐ I can name the five things that must be versioned, and I version all of them.
- ☐ My models are packaged with metadata, including the threshold.
- ☐ I track experiments, and I log the business metric alongside the statistical ones.

- ☐ My service validates inputs, returns its model version, and logs structured predictions.
  - ☐ I can choose between batch, online, shadow, canary, and blue-green, for a reason.
  - ☐ I monitor in three layers and know which layer catches which failure.
  - ☐ I can compute PSI and the KS statistic by hand, and I calibrate thresholds on my own data.
  - ☐ I know that input drift is not model decay, and I have a sampled ground-truth stream.
  - ☐ I have a retraining trigger written down, with guardrails and a regression check.
  - ☐ I have practiced a rollback and know how long it takes.
  - ☐ I can say where on the maturity ladder my company should stop.
- 

## Recap

---

- A model in production fails in four ways: **the world moves, nobody notices, nobody can reproduce it, nobody can roll it back.**
  - **"The model" is five things:** data, features, code, artifact, and config. A **registry** indexes the artifacts; the **threshold belongs in the metadata.**
  - **Track experiments** with parameters, metrics, and the artifact, and log the **business metric:** sorting by cost put a different run first than sorting by precision.
  - **Package** the model with metadata that answers "what is this and what was it trained on?". Treat artifacts as executables: pickles run code.
  - **Serve** batch, online, or streaming. Validate inputs, return the model version from `/health`, and log one structured line per prediction. The model was 0.2 ms of a 1.8 ms request.
  - **Deploy** with shadow and canary, and keep the old artifact one click away. Shadow mode showed Riverstone's candidate was right 17 times and wrong 16 on the parts where it disagreed: not a case for shipping.
  - **Monitor in three layers.** Ground truth is late, so pair a **fast proxy** (predicted-defective rate) with **slow truth** (a 2% audit of passed parts).
  - **PSI and KS** measure input drift cheaply. In Riverstone's data, PSI hit 4.4 while recall stayed above 0.94, and was flat while recall fell 15 points. **Input drift is not model decay.**
  - **Retrain** on a schedule and on events, with a held-out regression check, a business-metric floor, and a **re-tuned threshold.** Never train only on the model's own flags.
  - **Practise the rollback,** write the runbook, and make the post-mortem produce a monitor.
  - **Most companies should stop at level 2** of the maturity ladder and spend the saved effort on the audit sample.
- 

## Practice exercises

---

Work in `companion/ch56`, with the stream built by `simulate_production.py`.

## Warm-up

1. How many parts are there per week, and what is the overall defect rate? How does the rate change after week 16, and why?
2. Load the packaged model and print its metadata. Which three fields would you need most in an incident, and why?
3. Call `/health` and `/predict` with one good and one defective part. What does the response tell you that a bare probability would not?
4. Compute PSI for week 5 against weeks 0 to 7. Is it above or below the "stable" threshold, and what does that tell you about the threshold?

## Core

5. Compute recall per week for all 24 weeks and plot it. When does the fall begin, and how many weeks pass before it is obvious by eye?
6. Compute PSI on a *feature* rather than on brightness: take feature 0 and repeat the table. Does it tell the same story?
7. Add a monitor that compares the predicted-defective rate with the QC tally per week, and find the first week where the gap exceeds two percentage points.
8. Retrain including weeks 16 to 19, then find the threshold that minimizes Chapter 53's cost on weeks 20 to 23. How much does the optimal threshold move?
9. Write a CI check that loads the packaged model, scores the held-out Chapter 53 test set, and exits non-zero if recall falls below 0.90.
10. Send the service a request with a NaN in the features, and one with 108 zeros. What happens in each case, and which worries you more?

## Stretch

11. Implement shadow mode as a function that scores both models on a week and returns only the disagreeing part ids, sorted by how confident the disagreement is.
12. Add a second version of the artifact ( `defect_v2.joblib` ), switch the service to load whichever version an environment variable names, and time a rollback.
13. Simulate a third change: the camera is moved, so every image is shifted two pixels. Which monitor catches it, and how quickly?
14. Build a weekly digest: a small text report with service metrics, drift, the fast proxy, and the audit result, written to a file by a script you could schedule.

## Think about it (no code needed)

15. Your drift monitor has been amber for six weeks and the model is fine. What do you do?
16. Ground truth for missed defects arrives weeks late. Name three ways to shorten that loop, and what each costs.
17. The plant wants automated retraining with no human approval. What do you need in place first?
18. When would you argue *against* deploying a model at all, even one that works?

## Key terms

---

MLOps · model lifecycle · data versioning · feature versioning · model artifact · model registry · stage (staging, production, archived) · experiment tracking · run · parameter · metric · business metric · packaging · metadata · data hash · pickle risk · trusted types · batch serving · online serving · streaming serving · endpoint · input validation · health check · structured logging · big bang deployment · shadow mode · canary release · blue-green deployment · rollback · three-layer monitoring · service metrics · input monitoring · outcome monitoring · ground-truth delay · fast proxy metric · sampled audit · data drift · concept drift · population stability index · Kolmogorov-Smirnov statistic · alert fatigue · retraining trigger · regression check · threshold re-tuning · feedback loop · runbook · incident · post-mortem · MLOps maturity ladder · feature store

*(All terms are defined in the Glossary, Appendix A.)*

---

## Where this leads

---

- **Chapter 57, LLMOps**, applies all of this to language models, where the monitoring is harder because there is no accuracy to measure.
  - **Chapter 53, Deep Learning in Depth**, trained the model this chapter operates, and chose the threshold that section 56.9 re-tunes.
  - **Chapter 46, Pipelines & Orchestration**, schedules the batch scoring, the retraining, and the monitoring jobs.
  - **Chapter 47, Data Quality, Observability & Contracts**, is the same discipline one layer down, for the data feeding the model.
  - **Chapter 32, Analytics Engineering with dbt**, is where the CI habits came from; a model repository deserves the same treatment.
  - **Chapter 64, Responsible AI & Governance**, covers model cards, audit trails, and who is accountable when an automated decision is wrong.
  - **Chapter 74, Machine Learning & AI Question Bank**, has the interview questions, including "how would you know your model has gone stale?"
- 

## Answers to practice exercises

---

*(In the finished book these move to Appendix G.)*

1.

```
import numpy as np

stream = np.load("production_data/stream.npz", allow_pickle=True)
features, labels, week, kind = stream["features"], stream["labels"], stream["week"], stream["kind"]

print(f"parts per week: {int(np.bincount(week).mean())}, defect rate overall {labels.mean():.2%}")
print(f"weeks 0-15: {labels[week < 16].mean():.2%}")
print(f"weeks 16-23: {labels[week >= 16].mean():.2%}")
print(f"flash parts in weeks 16-23: {(kind[week >= 16] == 'flash').sum()}")
```

```
parts per week: 500, defect rate overall 8.77%
weeks 0-15: 8.19%
weeks 16-23: 9.95%
flash parts in weeks 16-23: 69
```

The rate rises by roughly two points from week 16 because the new mould adds a defect type on top of the existing ones. That is a **change in the world**, not a change in the model, and it is the kind of thing a plant knows on the day it happens: the cheapest monitor Riverstone has is the production meeting's change log.

2.

```
import joblib

metadata = joblib.load("models/defect_v1.joblib")["metadata"]
for key in ("version", "threshold", "trained_on", "data_hash", "trained_at", "metrics"):
    print(f" {key:<12} {metadata[key]}")
```

```
version      defect-v1
threshold    0.1
trained_on   ch53 defect_data (6,000 images, seed 53)
data_hash    1e86c528a4bfe346
trained_at   2026-03-02T09:00:00
metrics      {'recall': 0.95, 'precision': 0.95}
```

In an incident the three that matter are **version** (is the thing running the thing I think is running?), **threshold** (what decision boundary is in force?), and **trained\_on** with its hash (does this model know about the change we just made?). Everything else is useful later; those three are useful at 2 a.m.

3. The response carries `model_version`, `threshold`, and `latency_ms` alongside the probability. That turns every prediction into a self-describing record: when a QC inspector disputes a verdict three days later, the logged line says which model made it, under which threshold, and how long it took. A bare probability cannot be audited, and an audit trail you have to reconstruct is one you do not have.

4.

```
def psi(reference, recent, buckets=10):
    cuts = np.quantile(reference, np.linspace(0, 1, buckets + 1))
    cuts[0], cuts[-1] = -np.inf, np.inf
    a = np.clip(np.histogram(reference, cuts)[0] / len(reference), 1e-4, None)
    b = np.clip(np.histogram(recent, cuts)[0] / len(recent), 1e-4, None)
    return float(((b - a) * np.log(b / a)).sum())

brightness = stream["brightness"]
baseline = brightness[week < 8]
print(f"week 5 against the baseline: PSI {psi(baseline, brightness[week == 5]):.4f}")
print(f"the largest PSI among the stable weeks 0-7: "
      f"{max(psi(baseline, brightness[week == w]) for w in range(8)):.4f}")
```

```
week 5 against the baseline: PSI 0.0192
the largest PSI among the stable weeks 0-7: 0.0218
```

Week 5 is far below 0.1, and so is every stable week. **That is how you calibrate the threshold:** the largest value your own stable period produces is the noise floor, and an alert threshold should sit meaningfully above it. Importing 0.1 and 0.25 from a credit-scoring textbook without this check is how alert fatigue starts.

5. Recall sits between 0.94 and 1.00 for the first sixteen weeks, then drops into the 0.79–0.88 band. Plotted, the fall is visible almost immediately; in a weekly table with normal week-to-week noise, it takes three or four weeks before anyone would swear to it. That lag is the argument for **two** outcome signals: the fast proxy moves within a week, and the audited recall confirms it.

6. Feature 0 is the maximum blob response in the top-left region, and its distribution moves when the belt brightens, because the kernel responds to contrast. The story is similar but noisier than brightness. Two practical lessons: monitoring one interpretable summary (mean brightness) is often clearer than monitoring 108 features, and if you do monitor features, monitor the few the model actually relies on, which you can find from permutation importance (Chapter 38).

7.

```
from sklearn.metrics import confusion_matrix
import joblib

model = joblib.load("models/defect_v1.joblib")["model"]
first_gap = None
for w in range(24):
    rows = week == w
    predicted = (model.predict_proba(features[rows])[:, 1] > 0.1).mean()
    actual = labels[rows].mean()
    if first_gap is None and actual - predicted > 0.02:
        first_gap = w
    if w in (15, 16, 17, 18):
        print(f"week {w}: predicted {predicted:.1%}, actual {actual:.1%}, gap {actual - predicted:+.1%}")
print(f"\nfirst week where the gap exceeds two points: {first_gap}")
```

```
week 15: predicted 9.2%, actual 9.0%, gap -0.2%
week 16: predicted 7.8%, actual 9.2%, gap +1.4%
week 17: predicted 10.0%, actual 11.4%, gap +1.4%
week 18: predicted 7.8%, actual 8.6%, gap +0.8%
```

```
first week where the gap exceeds two points: None
```

The gap opens almost exactly when the new mould starts. This monitor costs nothing (a count the model already produces, and a tally QC already keeps) and it is the fastest label-free signal available. Put both lines on one chart and the failure announces itself.

8. Retrain on the original data plus weeks 16 to 19, then sweep thresholds on weeks 20 to 23 using ₹4,000 a miss and ₹40 a false alarm. The optimum moves **upwards** compared with `defect-v1`, because the retrained model is more confident about the defects it now knows, and a lower threshold buys false alarms without buying recall. The general rule: **every retrain invalidates the old threshold**, and re-tuning it is part of the release, not a follow-up ticket.

9.

```
#!/usr/bin/env python3
"""ci_model_check.py - fails the build if the packaged model falls below the agreed floor."""
import sys
import joblib, numpy as np
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

FLOOR = 0.90
bundle = joblib.load("models/defect_v1.joblib")
X = np.load("../ch53/defect_data/features.npy"); y = np.load("../ch53/defect_data/labels.npy")
_, X_test, _, y_test = train_test_split(X, y, test_size=0.25, random_state=53, stratify=y)
probabilities = bundle["model"].predict_proba(X_test)[: , 1]
tn, fp, fn, tp = confusion_matrix(y_test, probabilities > bundle["metadata"]["threshold"]).ravel()
recall = tp / (tp + fn)
print(f"recall {recall:.3f} (floor {FLOOR}), false alarms {fp}, model {bundle['metadata']}
['version']}")
sys.exit(0 if recall >= FLOOR else 1)
```

Run it in CI on every change to the model or the feature code (Chapter 32's pipeline, Chapter 29's exit codes). It is the model equivalent of a unit test, and it catches the most common release accident: shipping an artifact that was never evaluated at the threshold it will run under.

10. The NaN request is rejected with a 422 by the explicit `np.isfinite` check. The 108 zeros are accepted, because they are a perfectly valid input shape, and the model returns a confident "good".

**The second should worry you more.** Malformed data announces itself; *plausible* data that never occurs in reality (an all-zero frame from a camera that has lost its feed) is scored silently. Defenses: a range check per feature against the training data, and an input monitor that alerts on impossible summaries, such as a mean of exactly zero.

11. Return the disagreements sorted by `abs(probability_new - probability_old)`, largest first. That ordering is what makes human adjudication affordable: an inspector who can look at thirty parts should look at the thirty the two models disagree about *most*, because those carry the most information about which model is right. It is the same principle as active learning, and it turns a fixed review budget into the best possible evidence.

12. Store the version in an environment variable ( `MODEL_VERSION=defect-v2` ), load `models/{version}.joblib` at startup, and restart. Rollback is then `MODEL_VERSION=defect-v1` plus a restart, and the threshold returns with the artifact because it lives in the metadata. Time it. If the number is over five minutes, find out which step is slow *before* the incident, because during one the clock is being watched by people who are not reading your logs.
13. A two-pixel shift changes the convolution features noticeably (max-pooling absorbs some of it, which is exactly what pooling is for), so **input monitoring catches it quickly**, probably within a day. Recall may barely move, because the defects are still visible. That makes it the mirror image of the flash defect: drift high, performance fine. The right response is not to retrain but to **ask the plant what changed**, which is a phone call, and to log the answer so the next person knows.
14. The digest should fit on a phone screen: uptime and p95 latency, the PSI for one or two summaries with last week's value beside it, the predicted-defective rate against the QC tally, the audited recall if new audit results arrived, and the model version in force. Add one line at the top saying whether anything needs action. **A digest nobody reads is worse than no digest**, so write it for the plant manager, not for yourself, and put a name against it.
15. First, check whether the model is actually affected: score a recent labeled sample if you have one, or compare the predicted-defective rate with QC's tally. If performance is fine, **the alert is wrong, not the model**: recalibrate the threshold on your own stable period, and write down what changed (in Riverstone's case, new lamps) so the amber has an explanation attached. What you must not do is leave it amber, because in a month nobody will look at it, and the next alert will be the real one.
16. Three ways: **a sampled audit** of items the model passed (costs inspector time, and it is the one that actually measures missed defects); **a fast proxy** such as the predicted-positive rate or the distribution of predicted probabilities (costs nothing, measures change rather than correctness); and **a downstream signal** that arrives sooner than the final one, such as customer complaints logged at the depot rather than at the invoice stage (costs integration work). Most teams need all three, because each is cheap and blind in a different way.
17. Before automating the approval you need: an evaluation set the retrain cannot see, including old data; a **business-metric floor** that blocks the release; a regression check against the original data; automatic threshold re-tuning with its own floor; shadow or canary deployment with a rollback that has been timed; a data-quality gate so a corrupt week cannot become training data; and an audit log of what shipped, when, and on which data. Then automate, and keep a human approval for the first several releases anyway.
18. When the decision is rare enough that a person can make it well; when an error is catastrophic and the model's failure mode is silent; when the data needed to monitor it will never exist, so you could never tell whether it still works; when the rule is already known and writing it down is cheaper (Chapter 53's section 53.10); and when nobody will own it. That last one is the most common in practice: **a model with no owner degrades to a liability**, and the most valuable thing this chapter teaches is that the ownership, the monitoring, and the runbook are the deployment, not the extras.